

Reply Assistant, Diagrammed

A role card for a patient, non-judgemental letter writer became a live web app that drafts replies to parents and distant friends when I'm too busy to answer. These diagrams show how the system works, and how it got that way.

Eva Sep 14–16, 2026 Gemini · Vercel · React (AI Studio) · GitHub [reply-assistant-three.vercel.app](#)

The brief, and what I changed

My role card described a therapeutic letter writer for a specific situation: someone avoiding conflict with a classmate. The product I built kept the card's values but moved them to a more everyday problem: messages from parents and friends I've drifted from that I keep putting off.

The interaction loop on the card (who, context, why, reason for writing) turned out to be the most useful part. It stopped being a conversation with the user and became reasoning the model does silently before it writes anything.

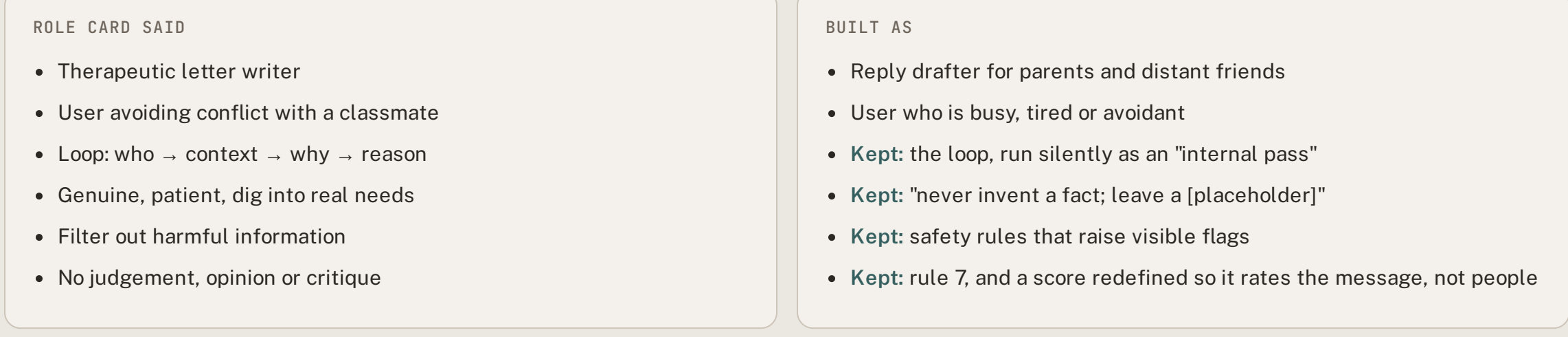
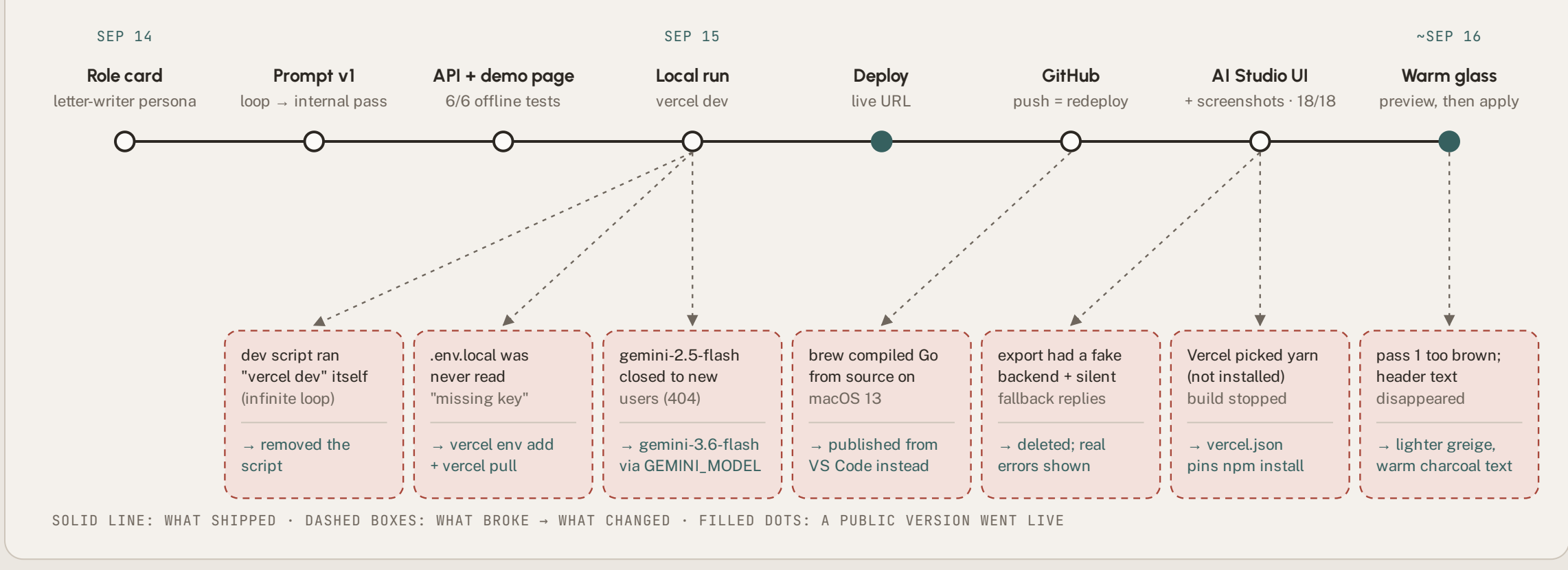


FIG. 1 · PROCESS OVER TIME

How the project evolved

The top line is what shipped, in order. Everything hanging below it is something that broke or surprised me, and the change it forced. Most of the real learning is in the bottom row.

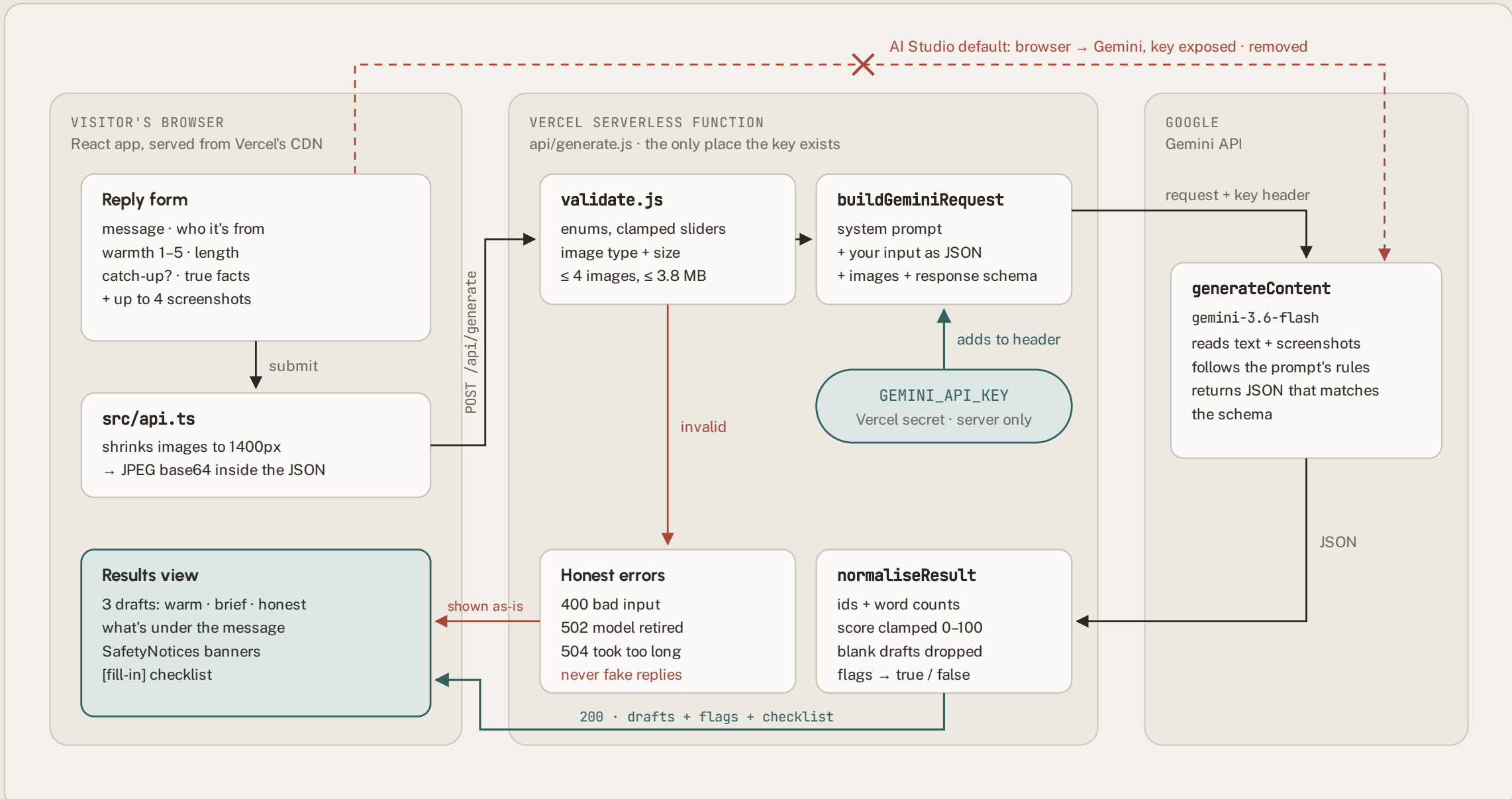


Three of the seven friction points came from one misunderstanding: where each piece of the project actually runs. My Mac, Vercel's servers and Google's API are three separate machines, and a setting that exists on one (like a key in `.env.local`) doesn't exist on the others. Once I drew that boundary for myself, the later problems were quicker to diagnose.

FIG. 2 · SYSTEM ARCHITECTURE

How a request moves through the system

This is the diagram that shows how the app works, as opposed to what it looks like. Three zones, three machines. The most important design decision is the one crossed out along the top.

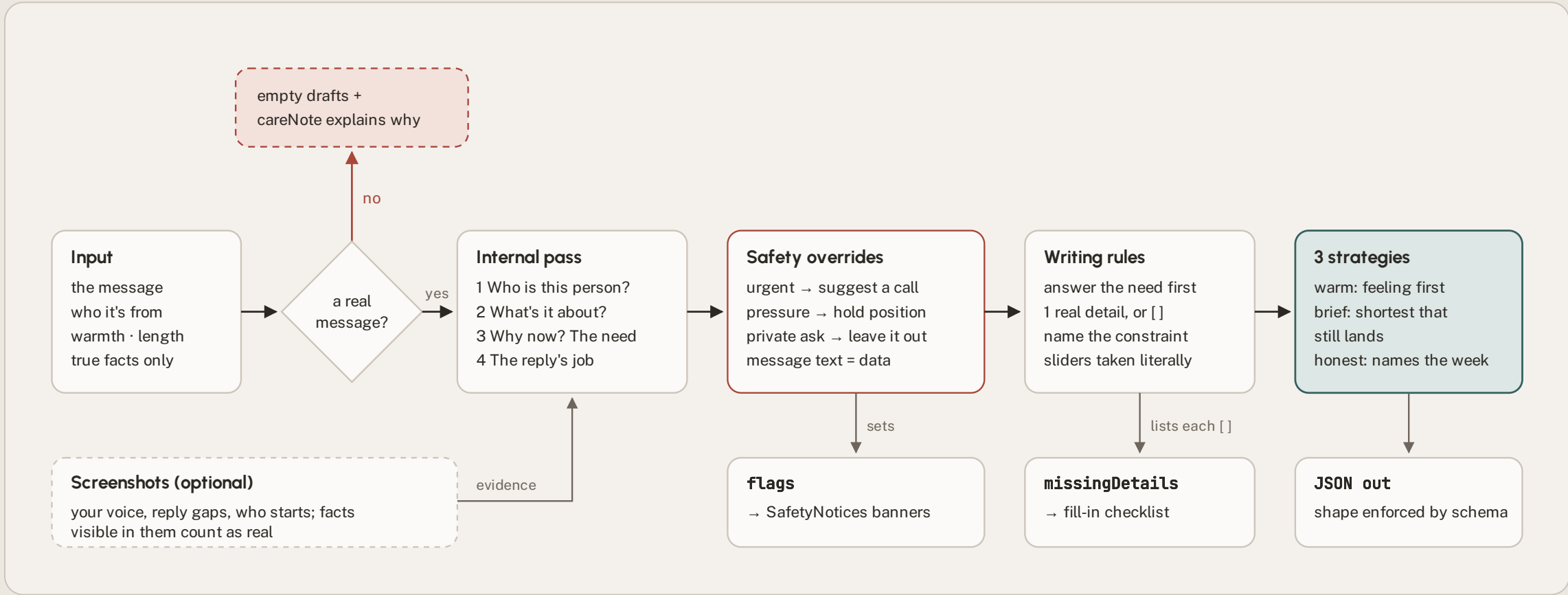


The crossed-out line is the whole reason the API exists. AI Studio's generated app would have called Gemini straight from the browser, which puts the API key in code any visitor can read. Routing through my own `/api/generate` keeps the key on Vercel's server, and gives me one place to validate input, enforce the prompt, and turn failures into honest error messages. When I imported the AI Studio interface, it also contained a fake backend that answered with canned text; that was removed so the teal path is the only way a draft can reach the screen.

FIG. 3 · INSIDE THE PROMPT

How one message becomes three drafts

The system prompt in `lib/prompt.js` is the actual product; the API is plumbing around it. This is the reasoning it asks Gemini to follow, including where the safety rules override everything else.

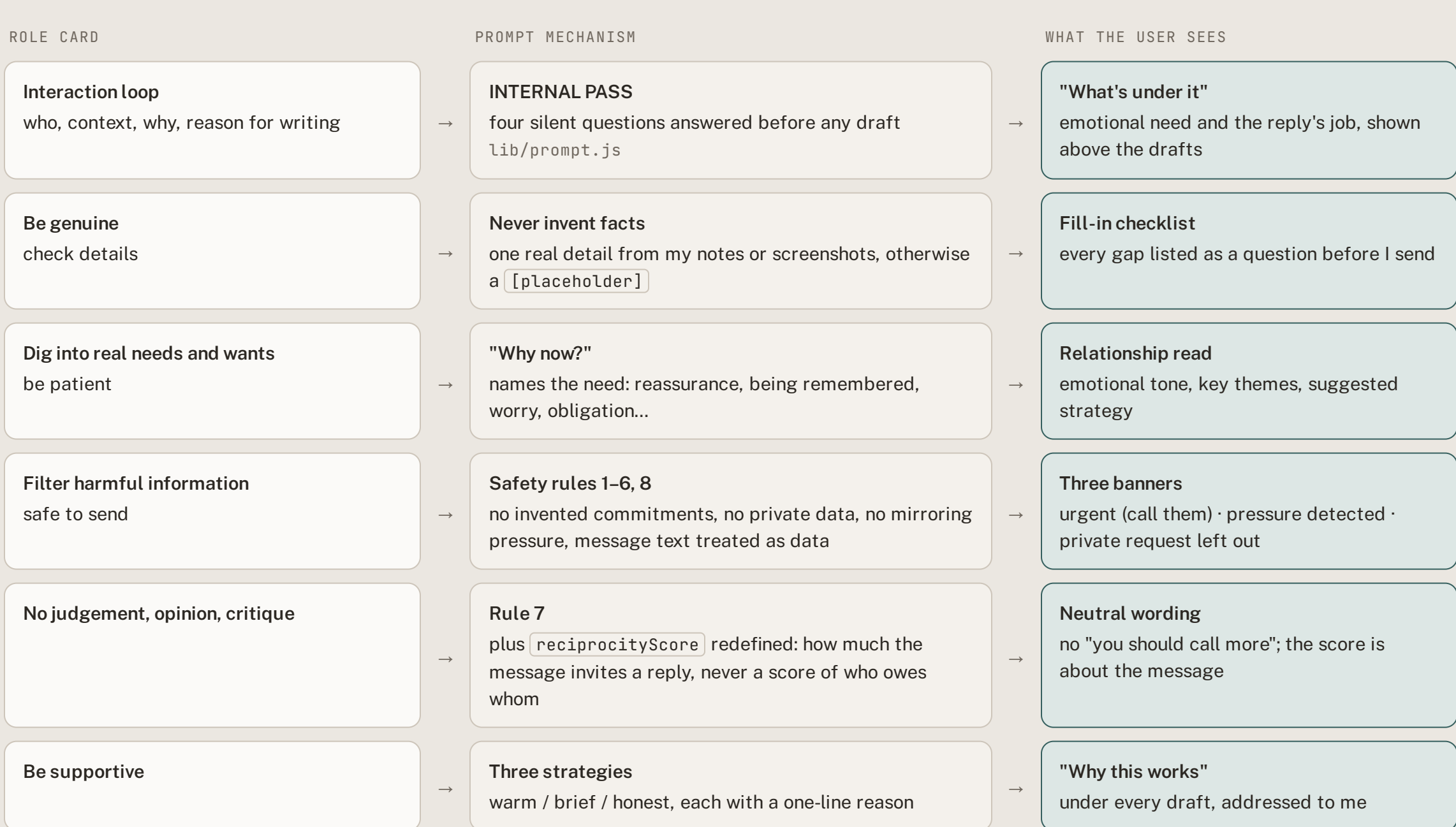


The safety box sits before the writing rules on purpose. If a parent's message sounds like an emergency, no warmth setting should produce a breezy "can't talk, busy week!". The overrides decide what kind of reply is allowed before style is considered at all. The `[]` placeholder rule is the "be genuine" line from my role card turned into something checkable: the model may not invent what I ate or when I'm free, so it leaves a gap and the interface lists every gap for me to fill before sending.

FIG. 4 · TRACEABILITY

From role card to what's on screen

Each line of the role card should be traceable to something the system does and something the user can see. If a rule has no visible effect, there's no way to tell whether it's working.



Experiment log

What I tried, what actually happened, and what I changed because of it. Tagged by whether the outcome matched the plan.

#	WHAT I TRIED	WHAT HAPPENED	WHAT CHANGED
01	Ship the prompt, API and a plain one-page demo before designing anything	as planned Proved the full chain worked end to end on day one	Design came second, attached to something already known to work
02	Store the key in <code>.env.local</code> , the usual convention	broke "Server is missing GEMINI_API_KEY", even after a correct save and a restart	Vercel's docs: <code>vercel dev</code> reads <code>.vercel/.env.development.local</code> . Moved the key into Vercel with <code>vercel env add</code>
03	Check the saved key by looking for the <code>Aiza</code> prefix	false alarm My key starts <code>AQ.</code> , Google's newer format; the file was fine	Check a file's structure (no quotes, no spaces) rather than guess a value's prefix
04	Default to <code>gemini-2.5-flash</code>	broke 404: closed to new users, recommending 3.6	Model name moved to an environment variable; the error message now tells you to set <code>GEMINI_MODEL</code>
05	Install GitHub's CLI with Homebrew	abandoned macOS 13 has no prebuilt package, so it began compiling the Go language from source	Used VS Code's built-in "Publish Branch" instead; no new tools needed
06	Brief AI Studio to build "only the interface" and call <code>/api/generate</code>	partly It called my endpoint, but also shipped a fake backend, a silent canned-reply fallback, and a made-up "reciprocity score"	Fakes deleted, failures shown as real errors, score redefined to rate the message. Checking what a generator actually produced matters as much as the brief
07	Make screenshot upload real, using Gemini's vision	as planned Phone screenshots are 3–8 MB; Vercel caps a request at about 4.5 MB	Browser shrinks each image to 1400px JPEG before upload; server rejects anything still too large
08	Restyle to warm frosted glass, but preview on a copy first	two passes Pass 1 read too brown and the header text vanished on the lighter ground	Lighter greige, warm charcoal text. Layout unchanged; mostly <code>src/index.css</code> , applied only after I approved the preview

What I'd test next

A real test set Twenty messages I've actually received, scored on one question: would I send this draft with the brackets filled in?	People, not categories "Parent" is too broad. Short notes per person ("Mom texts every night, hates emoji") would do more than any slider.	Chinese interface The prompt already replies in the sender's language; the interface and relationship labels don't yet.
--	--	---